

2. 搜索模块：机器人避障寻径问题

2.1 实验背景

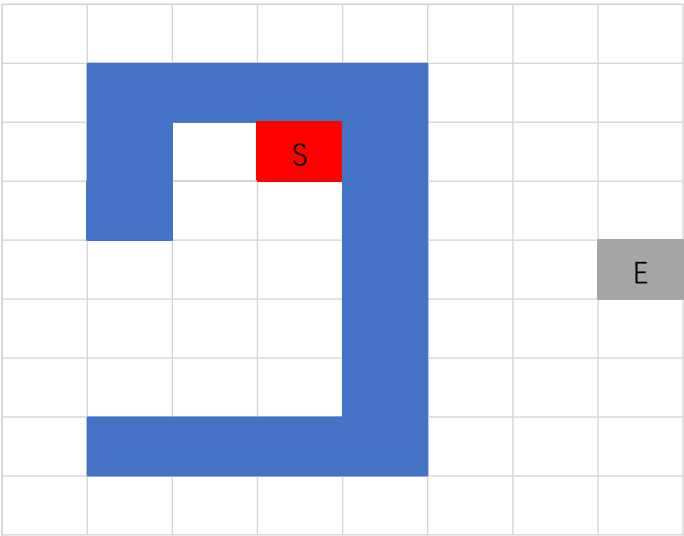
机器人自主导航是智能机器人技术体系中的核心研究方向之一，避障寻径作为自主导航的关键环节，直接决定了机器人在非结构化、动态复杂环境中的作业能力与安全性。

为系统探究不同路径规划算法的性能特点与适用场景，本实验以静态栅格地图下的机器人避障寻径为基础场景，设定固定起点（S）、终点（E）及不可通行障碍区域，要求实现从起点到终点的最短路径搜索。实验首先聚焦经典算法，要求实现 A* 与 BFS 算法的路径搜索，并通过可视化手段展示搜索过程，深入分析两种算法的原理、流程与性能差异；进一步拓展至高难度场景，探索强化学习、深度强化学习或遗传算法、人工势场算法等启发式算法的应用与改进方向；同时引入 Dijkstra 等其他算法作为补充对比，分析各类算法的局限性与优化空间。

2.2 实验内容

机器人避障寻径是机器人领域的一项重要技术，该技术要求实现机器人在复杂环境中的自主导航。在现实生活中，机器人的自主导航需要考虑各种因素，包括环境的复杂性、障碍物的位置和形状、机器人的运动能力等。如何解决这些问题是机器人避障寻径任务的研究难点。基于规则的传统路径规划方法，如 A*算法、Dijkstra 算法等，已经在机器人路径规划中得到广泛应用，并能够在大部分场景中实现较高的准确度。这些算法的核心思想是通过搜索算法在地图中找到最优的路径。这些算法具有计算复杂度低、实现简单等优点。

在本实验中，我们给定如下的机器人避障寻径场景图：



其中 S 为机器人寻径的起点，E 为寻径的终点，蓝色区域为无法通过的障碍。要求机器人从 S 点出发，绕开障碍，到达 E 点。本实验需要参与者自行设计核心算法规则完成机器人从起点到终点的最短路径搜索。

2.3 实验要求

- a) 已提供场景图的的矩阵表示，以及机器人起始点和终止点的坐标。要求找到从起始点到终止点的最短路径
- b) 禁止采用 hard code 的方式
- c) 可视化最短路径图搜索过程

要求细则：

- (1) 使用 A*和 BFS 算法，对起点到终点进行可视化搜索。并对两种算法流程、算法原理详细说明。
- (2) 在(1)的基础上，比较两种算法的差异、性能区别。
- (3) 自主设计更高难度场景图，并结合强化学习、深度强化学习，或使用其他的启发式算法（如遗传算法、人工势场算法）。并进行可能的改进。
- (4) 提出其他不同的解决方案（如 Dijkstra 等）。分析所设计算法的局限性，及改进点。
- (5) 最短路径查找过程的动态化展示。

2.4 实验环境

建议采用 python 来实现目标算法。允许调用基于 Python 的 Pandas、Numpy、Sklearn 等算法库。Pytorch 等深度学习框架也是被鼓励使用的。如果用到深度学习模型，需要搭建模型运行环境。为保证深度学习模型的正常运行，不同 python 库之间版本需要正确匹配。建议使用 anaconda 自动管理各种 python 库。

2.5 参考资料

Python:

<https://www.runoob.com/python3/python3-tutorial.html> Numpy:

<https://www.numpy.org> Sklearn:

<https://scikit-learn.org> Pytorch:

<https://pytorch.org/>